Neighborhoods, aggregation, etc.

Gabriel Arellano

2026-05-31

Contents

1 Context

Spatial neighborhood analyses are used to quantify how organisms, stems, species, functional groups, or other ecological units are arranged around focal individuals or locations. In forest ecology, these analyses are commonly used to evaluate things like:

- aggregation of conspecific individuals. Classical summaries include neighbor counts or densities within circles or rings, Ripley's K, and transformations such as Besag's L.

- amount of conspecifics and heterospecifics around focal trees, e.g. for density dependence analyses.
- spatial segregation of species.
- crowding indexes, as a proxy for competition.
- etc.

In some of these applications, but not all, the observations are compared with spatial null expectations. The most common form of null expectation for points is the Poisson point patter = uniform random and independent distribution of all points. Other null models generate expectations respecting the aggregation of different subsets of points (e.g. the torus translation test often applied for testing species-habitat associations).

2 Code

This tutorial introduces updated code and instructions for calculating neighborhood summaries at multiple spatial scales. In forest ecology, neighborhoods can be used for many questions and in many different ways. It is a type of analysis that requies high flexibility. It is not feasible to incorporate all that flexibility within functions. Therefore, the code is intentionally auxiliary in nature, not heavily wrapped. The functions are helpers to:

- calculate distances between points, but *very fast.*
- handling boundaries of arbitrary shape, not just rectangular windows: relevant for edge effects.
- contribute useful defaults and heuristics.

An important decision was to leave out of the functions the operations that require maximum flexibility in real-world ecological applications:

- defining classes or applying filters.
- aggregating / summarizing to get the metrics of interest, given a set of points in a neighborhood.
- generating null expectations and comparing them with observations.

IMPORTANT COROLARY: to apply this code to real ecological analyses, it is not possible to plug "data" in and forget. You would need to adapt the examples / instructions below to your own case.

As this is code for spatial analyses, it uses general inputs and general vocabulary of spatial analyses:

- `query` and `reference` inputs, as tables of (x, y) coordinates. By "query" it is meant the focal trees or locations, around which we find neighbors. By "reference" it is meant the (potential) neighbors. In some cases, reference = query, e.g. to study aggregation of conspecifics.
- boundaries as spatial `sf` polygon objects.
- `radii`: the sequence of scales of interest.
- `case`: either circles or rings, for different types of neighborhoods.

Load the code and inspect it here:

```
# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd())
path <- substr(getwd(), 1, g + attr(g, "match.length"))

# Source functionality:
source(paste0(path, "evaluation/0-code-to-source.r")) # auxiliary
f = paste0(path, "03_spatial/R/new/neighborhoods.R")
source(f)

# Check the functions
functions_in_script(f)
```

```
[1] "get_distances_up_to_r"     "get_boundary"
[3] "core_vs_edge"              "get_radii"
[5] "get_area_of_neighborhoods"
```

3 Workflow

The code takes responsibility for the boundary- and area-related difficult calculations and for the *very fast* identification of pairs of neighbors. Filtering and summarizing (e.g. counting neighbors, calculating crowding indexes, etc.) will be responsibility of the user. This section guides you through a general recommended workflow:

3.1 Step 1: Identify query and reference

Here you define what are the classes, how are you going to filter of subset the data, and so on. Remember: query = the focal trees or locations, and reference = the potential neighbors around the focal points.

Most importantly:

- spatial analyses in ecology almost invariably refer to individual trees, not stems. Make sure you are working with an individual-level table, not a stem-level table.
- spatial analyses will not work with missing coordinates, so you need to filter those out first.
- many forest datasets keep dead and alive trees together (in some ForestGEO tables, also the "prior" trees). Most spatial analyses focus on living trees, so make sure you filter by `status = "alive"` or the equivalent filter in your dataset.
- if your data exceeds your boundary, then you need to filter the data first: for example, if you have data from the full plot but want to run the spatial analyses within the (possibly complex) boundary of a single habitat within the plot.

Other than that, you have to define the query and reference datasets according to the goals of your analyses:

- species by species:
  - for example, define the structure of nested loops for sp $= i$ and sp $= j$, for species associations;
  - or j is any species different than i, for heterospecific counting;
  - of `reference` = `query` for conspecific aggregation analyses.
- by size classes; e.g. small trees around large trees, etc.
- other categories using other data, e.g. broken small trees (dead or alive) around large dead trees.

Alignment is very important, because the code will identify neighbors just by indexes in `query` and `reference` tables. To use those indexes to obtain summaries, etc., you need to respect the alignment with the table where you store data. For that, something like this is generally safe:

```
keep <- !is.na(data$x) & !is.na(data$y)
query_data <- data[keep,]
result <- get_distances_up_to_r(query = query_data[,c("x", "y")], ...)
```

While something like this is almost always a bad idea, as it would immediately break alignment:

```
result <- get_distances_up_to_r(query = na.omit(data[,c("x", "y")]), ...)
```

So, overall, the data processing step should filter rows (observations) but keep columns (variables of potential interest), and just call the {x, y} coodinates when needed. Something like this:

```r
# Subset data explicitly for the query points
keep1 <- !is.na(data$x) & !is.na(data$y) # mandatory for spatial analyses
keep2 <- data$sp == "PREMON" # if query is one species
keep3 <- data$dbh >= 10 # if query contains large trees, etc.
use_for_query <- keep1 & keep2 & keep3 & ... # all conditions that apply

query_data <- data[which(use_for_query),,drop = FALSE]

# Subset data explicitly for the reference points
keep1 <- !is.na(data$x) & !is.na(data$y) # mandatory for spatial analyses
keep2 <- data$sp %in% names(which(table(data$sp) >= 100)) # if reference contains common spp
keep3 <- data$dbh > 0 & data$dbh < 10 # if we want only small trees as neighbors, etc.
use_for_reference <- keep1 & keep2 & keep3 & ... # all conditions that apply

reference <-data[which(use_for_reference),, drop = FALSE]

# Then call the coordinate columns only when needed
neighbors <- get_distances_up_to_r(query = query_data[,c("x", "y")],
                                   reference = reference_data[,c("x", "y")],
                                   ...)
```

3.2 Step 2: Define boundary and scales of interest

Boundaries are needed to account for edge effects in spatial analyses. Most often, they are rectangular. However, the code here was developed to handle irregular boundaries, and be as general as possible. It could be used to run analyses within specific habitats, or for forests that do not completely fill the rectangular plot (e.g. Ngel Nyaki, Nigeria).

The `get_boundary()` function is a general helper that can transform a rectangular window into an `sf` object or estimate the area of observation based on a set of {x, y} points.

Similarly, the `get_radii()` function can return a vector of scales of interest, using rules of thumb, in case you do not have a specific vector of interest. It has a useful default for the maximum distance so, in practice, you would have to specify the `case` (circles or rings for the neighborhoods) and the number of scales to evaluate, which typically depends on the size of the datasets and constrain total code running times.

3.3 Step 3: Identify the neighbors

After defining query and reference datasets, boundary, and scales of observation, the `get_distances_up_to_r()` function will return all {i, j} pairs and their distance, up to a given maximum distance. Individuals i come from `query` and individuals j from `reference`.

These are just row indexes, so it is important that you keep alignment with the variables of interest, to be able to characterize the neighborhoods in the next step.

In general, given a set of scales of interest (`radii`), this is the most important thing you need to identify all neighbors at all scales:

```
neighbors <- get_distances_up_to_r(..., radius = max(radii))
```

It will return information info for the largest possible neighborhood, very fast, without the need to repeat calculations or store multiple results.

Note: in many applications, points in `query` are also included, completely or partially, within `reference`. This will result in {i, j=i} pairs, or neighbors of themselves. The most obvious case is when reference = query, for the study of conspecific neighbors.

In these cases, something like this will be required:

```
# Assuming "ID" is a unique point ID that applies to both datasets:
ID_in_query     <- query_data$ID[neighbors$query_i]
ID_in_reference <- reference_data$ID[neighbors$reference_j]
not_themselves  <- ID_in_query != ID_in_reference
neighbors       <- neighbors[not_themselves,]

# Only if reference = query this will work directly:
keep <- which(neighbors$query_i != neighbors$reference_j)
neighbors <- neighbors[keep,]
```

If, for whatever reason, you have used a stem-level table instead of an individual-level table, this would also get rid of all {i, j} pairs that belong to the same individual (assuming stems within individuals share coordinates):

```
keep <- which(neighbors$dist > 0)
neighbors <- neighbors[keep,]
```

From this large `neighbors` object it is easy to obtain the set of neighbors for each of the scales, doing something like:

```
neighbors <- get_distances_up_to_r(..., radius = max(radii))

for(k in 1:length(radii))
{
  # for circles
```

```
  radius.k = radii[k]
  w <- which(neighbors$dist <= radius.k)
  neighbors.k <- neighbors[w,]

  # for rings
  outer.radius.k = radii[k]
  if(k == 1) inner.radius.k = 0
  if(k >= 2) inner.radius.k = radii[k - 1]
  w <- which(neighbors$dist > inner.radius.k & neighbors$dist <= outer.radius.k)
  neighbors.k <- neighbors[w,]
}
```

Note that `get_distances_up_to_r()` is *very fast*, but returns huge objects. In preliminary runs, it returns 1-10 million rows per second. This is unavoidable, and there is not a more efficient way of reporting distances between neighbors than the sparse representation that this function returns. But, in some cases, the output can outflow memory. If that can happen, the general recommendation is to split the query dataset into chunks. The general template is:

```
# Get a very rough idea of the size of the output, using
# the average density of points in reference to guess
# the number of {i, j} pairs in the largest neighborhood.
n = nrow(query_data)
m = nrow(reference_data)
A = sf::st_area(boundary) # boundary as sf object, use get_boundary(...) if needed
a = pi*max(radii)^2 # assumes same units as boundary
Q = n * a  * m / A

# If Q is greater than few hundred millions, safer to split in chunks.
# The maximum allowed by data.table is 2^31 - 1 rows.
max_output_size = 10^8 # not a magic number, reduce it if needed
chunk_size = max_output_size / a / m * A
chunks <- split(1:n, ceiling(1:n / chunk_size))

# Go chunk by chunk
for(h in 1:length(chunks))
{
  # Subset the query only, not the reference
  # even if the reference = query.
  query.data.h <- query_data[chunks[[h]],, drop = FALSE]

  # Do things for this chunk only:
  # Note: the "reference" remains the same! You want to identify
```

```
  # all neighbors around the query individuals, not just some of them.
  neighbors.h <- get_distances_up_to_r(query = query.data.h[,c("x", "y")],
                                       reference = reference_data[,c("x", "y")],
                                       ...)

  # The i indexes in the {i, j} are internal to the chunk now!
  # It depends on how you end organizing the code, but
  # probably a good / safe practice is to go back to global query
  # indexes as soon as possible, with direct replacement:
  global.query.i <- chunks[[h]][neighbors.h$query_i]
  neighbors.h$query_i <- global.query.i

  # Etc.

}
```

Loops may be needed for the chunks, because of the storage limitations. The need for other loops (species by species, etc.) will depend on how one summarizes or aggregates the metrics of interest.

3.4 Step 4: Characterize neighborhoods with the metric(s) of interest

At this point, and regardless of the structure into loops, you should have pairs of individuals that are neighbors at a given scale. Then, you can process this information in any way you want:

- count number of neighbors
- count conspecific vs. heterospecific neighbors
- calculate neighborhood diversity
- calculate accumulated biomass
- calculate crowding indexes, possibly weighting by distance
- etc.

Two things are important:

First, and obviously: you should have used the appropriate `reference` input. If you want to count number of species in the neighborhoods you should have used a `reference` input with all the species, not just one, and so on.

Second: alignment is key. The output of `get_distances_up_to_r()` are pairs of query and reference indexes as {i, j}, and these are rows in matrixes that only contain (x, y) coordinates. If you want to call other variables (basal area, biomass, species identity, etc.) then these indexes must correspond to the tables with those variables. For that reason, the safe way of

obtaining neighbors is to have complete data tables for the query and reference points, and call the {x, y} columns when necessary, without breaking alignment in any way. See Step 1.

The code was developed specifically to maximize flexibility in this step. There is no room here to even list the possible ways the information can be summarized. Neighborhoods are just small pieces of forest, local communities, and you can apply to them, in theory, almost any ecological analysis. In practice, you would be constrained by computational limitations, as neighborhoods are very numerous, so they are summarized by relatively simple metrics (most often, just counting individuals).

Just a few reminders:

- `aggregate()` is base R and useful for simple grouped summaries without extra dependencies. It is explicit and stable, but can become verbose when several summaries or several grouping variables are needed.
- `dplyr` provides readable verbs such as `group_by()`, `summarise()`, and `count()`, often connected with the pipe operator `|>`. This syntax is usually easier to read and modify, and is very well documented in many sources.
- The output of `get_distances_up_to_r()` is a data.table. The approach to summarize/aggregate data.table objects is almost guaranteed to be the fastest and most memory-efficient option for large tables, using compact expressions such as `DT[, .(n = .N), by = group]`. This syntax is not widely known, so this is probably best suited for advanced users or when performance is critical. See Example 3 below.

If you only want to count neighbors, in simple cases, `tabulate()` is better than `table()` because it would end aligned, including zero counts:

```
d <- get_distances_up_to_r(query, reference, ...)
nn <- d[d$dist <= radius,] # scale of interest
counts <- tabulate(nn$query_i, nbins = nrow(query))

# table(nn$query_i) # this would require further processing
```

3.5 Step 5: Contrast with expectations

Calculating areas

At the very minimum, you need to calculate the areas of the neighborhoods. Areas will be well-known and easy to derive from radii for the core areas far from the boundary, but will be smaller and smaller the closer the focal point is to the (possibly complex) boundary. The `get_area_of_neighborhoods()` function does that, for any boundary.

Having the available area for each neighborhood, and then expressing metrics on a per area basis, is one explicit way of comparing observations with expectations, under the assumption that the metric of interest will scale directly as a function of the area. This is true for individual counts, accumulated basal area, etc., but it won't be true for other metrics of interest.

The `get_area_of_neighborhoods()` function includes useful defaults for boundary and radii. Thus, if you do not have that pre-defined, you could run this function first, as part of Step 2. It does not require neighbor information, just the location of the query points.

Numerical null models

The alternative to expressing things on a per-area basis is to run numerical null models.

Null models compare observations (one instance) with expectations (many instances). If the observations deviate a lot from expectations, then there is a strong pattern. This is more general, but also more time consuming. It is needed if one is using a metric that does not scale linearly with the area, such as averages or anything related to species diversity, etc.

Null models do not require the calculation of areas to control for edge effects. The null expectations already incorporate the fact that points near the boundaries will tend to have fewer neighbors (and thus lower basal area, fewer species, etc). If deviations are standardized, they will solve for edge effects.

Often, deviations from expectations are linked to null hypothesis testing, so we can tell at what scales a given species is *significantly* more aggregated than expected by chance, at what scales it is roughly as aggregated as expected by chance, etc. If you are testing hypotheses, you probably want to control for Type I error rate, because you are testing many scales, and (possibly) many species. It is beyond the scope of this tutorial to explain how to do that, but, as a general rule: do better than Bonferroni.

The examples below implement the simplest of the null models: Poisson point pattern, or uniformly random and independent distribution of points. This is the null model implicit in most standard spatial analyses, and the one to use when studying aggregation. For other metrics (for example, abundance of species A around species B, number of species around focal locations, etc.) the general recommendation is to use null models that are more respectful towards reality (e.g. torus translations).

Other

Code for spatial analyses often comes with options to test for hypotheses at different scales, under different assumptions. For example, `envelopes()` in `spatstat` for Ripley's K, or wavelet analysis.

4 Examples

This section uses real data to show how to run some spatial analyses in practice. It puts together all the individual steps mentioned above, with some variations. It includes:

- Example 1: Aggregation within one species, using number of individuals or accumulated basal area
- Example 2: Quantity of conspecifics and heterospecifics at short distances
- Example 3: Diversity in the neighborhood of different species

Note that, filtering, categorizations, summary metrics, and null expectations, are all outside the functions themselves!

```
# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd())
path <- substr(getwd(), 1, g + attr(g, "match.length"))

# Load data from the Luquillo forest plot, public domain, as an example
load(paste0(path, "data_examples/luquillo.full2.rdata"))
data <- luquillo.full2
dim(data)
```

```
[1] 125446     19
```

```
head(data)
```

```
   treeID stemID     tag StemTag     sp quadrat gx gy  DBHID CensusID dbh  pom
1       1      1 -110496 -110496 CECSCH     622 NA NA 103376        2  NA <NA>
3       2      2  -11392  -11392 PSYBER     503 NA NA 103377        2  NA <NA>
5       3      3 -114036 -114036 PSYBER    1223 NA NA 103378        2  NA <NA>
7       4      4  -11756  -11756 CECSCH     901 NA NA 103379        2  NA <NA>
9       5      5  -12584  -12584 CECSCH     801 NA NA 103380        2  NA <NA>
11      6      6  -15558  -15558 UNIDEN     303 NA NA 103381        2  NA <NA>
   hom  ExactDate DFstatus      codes nostems status  date
1   NA 1996-07-03     dead MAIN;DEADT       1      D 13333
3   NA 1995-02-09     dead MAIN;DEADT       1      D 12823
5   NA 1996-08-02     dead MAIN;DEADT       1      D 13363
7   NA 1995-02-23     dead MAIN;DEADT       1      D 12837
9   NA 1995-02-13     dead MAIN;DEADT       1      D 12827
11  NA 1995-02-03     dead MAIN;DEADT       1      D 12817
```

```
colnames(data)
```

```
 [1] "treeID"    "stemID"    "tag"       "StemTag"   "sp"        "quadrat"
 [7] "gx"        "gy"        "DBHID"     "CensusID"  "dbh"       "pom"
[13] "hom"       "ExactDate" "DFstatus"  "codes"     "nostems"   "status"
[19] "date"
```

```
summary(data)
```

```
    treeID           stemID            tag             StemTag
 Min.   :     1   Min.   :     1   Min.   :-172721   Min.   :-114036
 1st Qu.: 31363   1st Qu.: 29528   1st Qu.:  41423   1st Qu.:  28280
 Median : 62726   Median : 57530   Median :  84202   Median :  66158
 Mean   : 62726   Mean   : 57731   Mean   :  84106   Mean   :  64320
 3rd Qu.: 94089   3rd Qu.: 86554   3rd Qu.: 124460   3rd Qu.:  97486
 Max.   :125450   Max.   :114032   Max.   : 175926   Max.   : 146191
                  NA's   :30956                      NA's   :30956
     sp              quadrat            gx              gy
 Length:125446      Min.   : 101    Min.   :  0.01   Min.   :  0.0
 Class :character   1st Qu.: 522    1st Qu.: 86.43   1st Qu.:127.7
 Mode  :character   Median : 922    Median :164.29   Median :265.2
                    Mean   : 920    Mean   :163.49   Mean   :254.9
                    3rd Qu.:1316    3rd Qu.:243.74   3rd Qu.:380.8
                    Max.   :1625    Max.   :319.98   Max.   :499.9
                    NA's   :30956   NA's   :83870    NA's   :83870
    DBHID           CensusID          dbh              pom
 Min.   :103376   Min.   :2       Min.   :   5.40   Length:125446
 1st Qu.:132902   1st Qu.:2       1st Qu.:  15.70   Class :character
 Median :160900   Median :2       Median :  27.50   Mode  :character
 Mean   :161103   Mean   :2       Mean   :  63.61
 3rd Qu.:189924   3rd Qu.:2       3rd Qu.:  79.00
 Max.   :217400   Max.   :2       Max.   :1515.00
 NA's   :30956    NA's   :30956   NA's   :58283
     hom          ExactDate           DFstatus            codes
 Min.   :0.01    Length:125446      Length:125446      Length:125446
 1st Qu.:1.30    Class :character   Class :character   Class :character
 Median :1.30    Mode  :character   Mode  :character   Mode  :character
 Mean   :1.31
 3rd Qu.:1.30
 Max.   :3.90
 NA's   :58288
```

```
    nostems          status               date
 Min.   : 1.000   Length:125446      Min.   :12717
 1st Qu.: 1.000   Class :array       1st Qu.:12984
 Median : 1.000   Mode  :character   Median :13205
 Mean   : 1.207                      Mean   :13152
 3rd Qu.: 1.000                      3rd Qu.:13310
 Max.   :29.000                      Max.   :14013
 NA's   :30956                       NA's   :30956
```

```
# Spatial analyses almost invariably refer to individuals, not stems,
# so we should go from stem-level information to individual-level information.

# In real applications, you should pay attention and do proper data aggregation
# from the stem level to the individual level. That is more difficult than
# it seems. I do not want to over-complicate the example here, so I simply
# subset the dataset for the main stems of each individual, taking
# advantage of the codes. This is something that was important in the old
# workflows but NOT a general recommendation.

main <- grep("MAIN", data$codes)
data <- data[main,]
dim(data)
```

```
[1] 92456    19
```

4.1 Example 1: Aggregation within species

This example uses the new code to calculate aggregation of conspecific individuals. It is slower than alternatives like `spatstat::Ktest()` but it offers greater flexibility in the definition of boundary (complex) and the abundance/summary metric (outside the functions themselves).

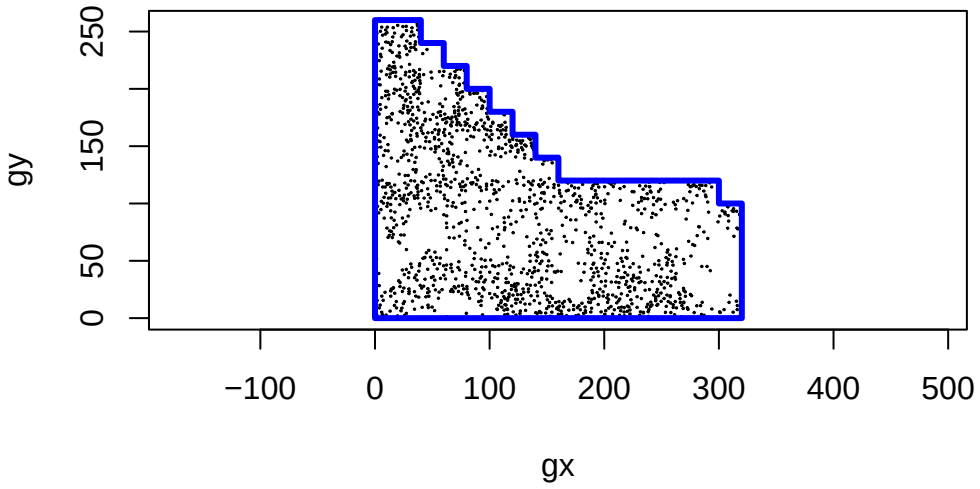
```
library(sf)
```

```
Warning: package 'sf' was built under R version 4.4.3
```

```
Linking to GEOS 3.14.1, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```r
# Study aggregation of one species, but only within a low-disturbance
# portion of Luquillo plot, using a boundary of complex shape:

low_disturbance_xy <- rbind(
  c(0,   0),
  c(320, 0),
  c(320, 100),
  c(300, 100),
  c(300, 120),
  c(160, 120),
  c(160, 140),
  c(140, 140),
  c(140, 160),
  c(120, 160),
  c(120, 180),
  c(100, 180),
  c(100, 200),
  c(80,  200),
  c(80,  220),
  c(60,  220),
  c(60,  240),
  c(40,  240),
  c(40,  260),
  c(0,   260),
  c(0,   0)
)

boundary  <- sf::st_sfc(st_polygon(list(low_disturbance_xy)))

# In general it will be useful to get subsets of the dataset with all the variables,
# so it is easier to align from a given point to a given value in the table.
sp = "PREMON"
keep1 <- data$sp == sp
keep2 <- data$status == "A"
keep3 <- !is.na(data$gx)
keep4 <- !is.na(data$gy)
keep  <- keep1 & keep2 & keep3 & keep4
query_data <- data[keep,]

pts <- sf::st_as_sf(query_data[,c("gx", "gy")], coords = 1:2)
inside <- lengths(sf::st_intersects(pts, boundary)) > 0
```

```r
query_data <- query_data[inside,]

# (In general, we would do the same for a reference subset, but here we are
# exploring conspecific aggregation, so query = reference).

plot(query_data[,c("gx", "gy")], asp = 1, cex = 0.1)
plot(boundary, asp = 1, border = "blue", lwd = 3, add = TRUE)
```




```r
# Scales to study?
radii <- get_radii(query = query_data[,c("gx", "gy")], m = 50, case = "circles")
max(radii)
```

```
[1] 64.3575
```

```r
# Identify the neighbors:
# We do it once, for the maximum radius. We subset and process later.
# Because we want aggregation of one species with itself, then reference = query.
neighbors <- get_distances_up_to_r(query = query_data[,c("gx", "gy")],
                                   reference = query_data[,c("gx", "gy")],
                                   radius = max(radii))
dim(neighbors)
```

```
[1] 374331      3
```

```
head(neighbors)
```

```
   query_i reference_j     dist
     <int>       <int>    <num>
1:       1           3 4.960262
2:       1        1398 4.780387
3:       1           2 3.769695
4:       1         811 3.491848
5:       1         810 3.861204
6:       1           1 0.000000
```

```
# Note: because reference = query in this application, every individual
# appears as a neighbor of itself. We can remove them:
keep <- which(neighbors$query_i != neighbors$reference_j)
neighbors <- neighbors[keep,]

# Then, we can summarize those neighborhoods any way we want.
# The data.table synthax is faster and, if you can, do it that way.
# There are different tools for data aggregation in R, and
# you may have to explore them, depending on your goals, size of dataset, etc.
# Here I use simple base code as a template, going scale by scale.

# First, I externalize the calculation of basal area
# (this would be done on "reference" in general, but
# for conspecifics reference = query)
query_data$BA <- pi*(query_data$dbh/2)^2
neighbors$BA_j <- query_data$BA[neighbors$reference_j]
head(neighbors)
```

```
   query_i reference_j     dist      BA_j
     <int>       <int>    <num>     <num>
1:       1           3 4.960262 22698.007
2:       1        1398 4.780387 12076.282
3:       1           2 3.769695 20106.193
4:       1         811 3.491848 17203.361
5:       1         810 3.861204  8171.282
6:       1         809 5.868219 10386.891
```

```
# Then, I create empty objects to store results:
N.total.in.neighborhoods <- matrix(NA, nrow = nrow(query_data), ncol = length(radii))
```

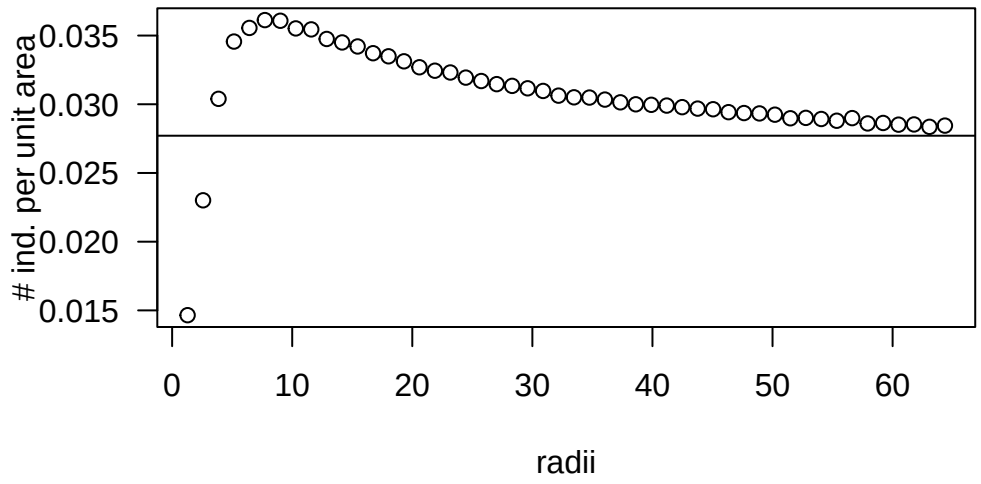
```r
BA.total.in.neighborhoods <- matrix(NA, nrow = nrow(query_data), ncol = length(radii))

# Then, I look through scales:
# (I use "k" to avoid confusion with {i,j} pairs of neighbors)
for(k in 1:length(radii))
{
  radius = radii[k]
  sub <- neighbors[neighbors$dist <= radius,]

  # Counting: how many times "i" appears = number of j neighbors
  counts <- tabulate(sub$query_i, nbins = nrow(query_data))
  N.total.in.neighborhoods[,k] <- counts

  # Sum basal areas:
  agg.ba <- aggregate(x = sub$BA_j,
                      by = list(sub$query_i),
                      FUN = "sum", na.rm = TRUE)
  BA.total.in.neighborhoods[agg.ba[,1], k] <- agg.ba[,2]

  # Track progress:
  #print(paste0(k, "/", length(radii)))
}

N.total.in.neighborhoods[1:5, 1:5]
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    0    3    5    6
[2,]    0    0    1    3    3
[3,]    0    1    3    4    5
[4,]    0    1    1    2    4
[5,]    0    0    0    2    3
```

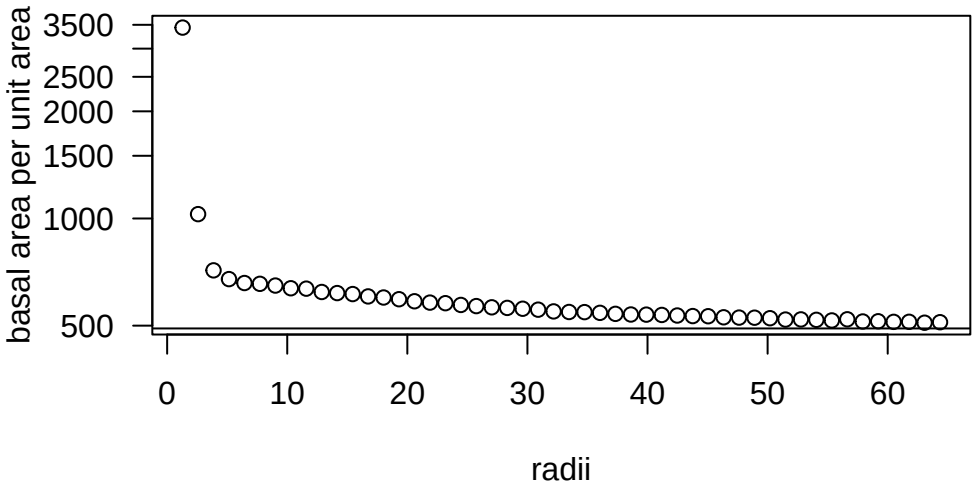
```r
BA.total.in.neighborhoods[1:5, 1:5]
```

```
     [,1]     [,2]     [,3]     [,4]     [,5]
[1,]   NA       NA 45480.84 80255.13 90642.02
[2,]   NA       NA 25446.90 56392.37 56392.37
[3,]   NA 10386.89 30634.46 56081.36 74950.55
[4,]   NA 10751.32 10751.32 27037.33 53540.59
[5,]   NA       NA       NA 27954.68 46823.87
```

```r
# Express in a per-area basis removes edge effects,
# and the effect of larger outer circles too.
# (I accept 5% error in the estimation of area of edge neighborhoods).
A <- get_area_of_neighborhoods(query = query_data[,c("gx", "gy")],
                               boundary = boundary,
                               radii = radii,
                               case = "circles",
                               tol = 0.05)


N.per.area.in.neighborhoods <- N.total.in.neighborhoods / A$areas
BA.per.area.in.neighborhoods <- BA.total.in.neighborhoods / A$areas

y <- colMeans(N.per.area.in.neighborhoods, na.rm = TRUE)
plot(radii, y, las = 1, ylab = "# ind. per unit area")
ref = nrow(query_data) / sf::st_area(boundary)
abline(h = ref) # the overall density of individuals
```



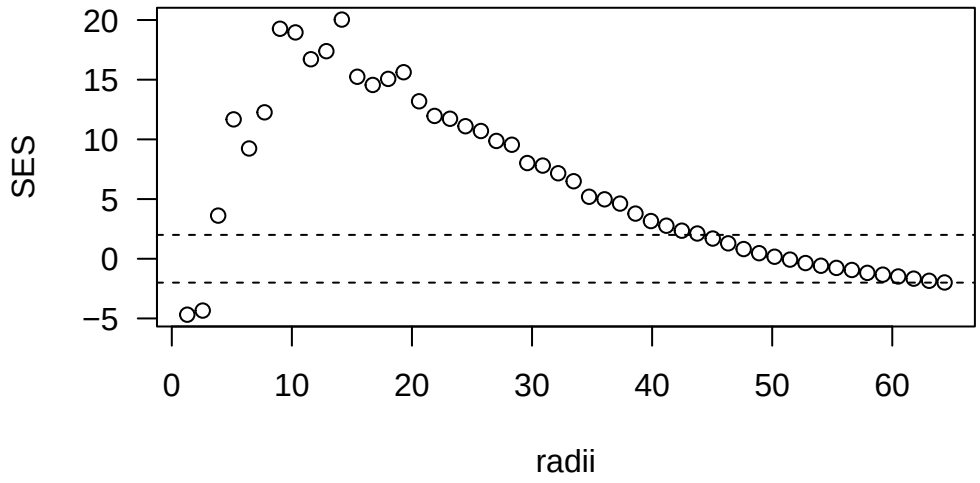

```r
y <- colMeans(BA.per.area.in.neighborhoods, na.rm = TRUE)
plot(radii, y, las = 1, ylab = "basal area per unit area", log = "y")
ref = sum(query_data$BA, na.rm = TRUE) / sf::st_area(boundary)
abline(h = ref) # the overall density of basal area
```



Having the available area for each neighborhood, and then expressing metrics on a per area basis, can be one explicit way of accounting for edge effects. This was the approach taken in old CTFS code. Alternatively, one can control for edge effects using numerical null models, and avoiding the calculation of areas entirely. The null expectations already incorporate the fact that points near the boundaries will tend to have fewer neighbors (and thus lower basal area, fewer species, etc). If deviations are standardized, they will solve for edge effects. I present a template on how to do that, for the simple case of number of neighbors. However, this would be most useful for metrics that do *not* grow linearly with area, like species diversity, and that cannot be directly corrected by expressing in a per-area basis.

```
# Null models compare observations (one instance) with expectations (many instances).
# If reality does not deviate much from the expectations by the null model, then
# the researcher cannot reject the null hypothesis modelled by the null model.
# If reality deviates  from the expectations, then we can reject the null hypothesis.

# Let's say we are interested in the mean number of conspecific
# neighbors at different scales, and whether that is more or
# less than the expected by chance.
obs <- colMeans(N.total.in.neighborhoods) # observations
#barplot(obs)

# The expectations will be stores in a matrix, with one row per simulation:
nsim = 9 # 999 or so for real applications -- small numbers for examples and rendering
null <- matrix(NA, ncol = length(radii), nrow = nsim)

# Get the null expectations
for(sim in 1:nsim)
{
  # The simplest of the expectations is uniform random distribution of points,
```

```
  # within the valid boundaries. Easy to get from sf::st_sample.
  # You may need more sophisticated null models here, depending on your goals.
  null_xy <- sf::st_sample(x = get_boundary(boundary),
                           size = nrow(query_data),
                           type = "random",
                           exact = TRUE)

  null_xy <- sf::st_coordinates(null_xy)

  # Expected conspecific neighbors, under the null distribution of points.
  null_neighbors <- get_distances_up_to_r(query = null_xy,
                                          reference = null_xy,
                                          radius = max(radii))

  keep <- which(null_neighbors$query_i != null_neighbors$reference_j) # remove neighbors of t
  null_neighbors <- null_neighbors[keep,]

  # Then same loop through scales as before:
  for(k in length(radii):1)
  {
    radius = radii[k]
    null_sub <- null_neighbors[null_neighbors$dist <= radius,]
    null_counts <- tabulate(null_sub$query_i, nbins = nrow(null_xy))
    null[sim,k] <- mean(null_counts) # store in the right place
  }
  #print(sim)
}

# There are different ways of comparing observations with expectations.
# A common one in ecology is the Standardized Effect Size calculated
# as SES = (observations - mean(expectations)) / sd(expectations).
mean.null <- apply(null, 2, mean, na.rm = TRUE)
sd.null <- apply(null, 2, sd, na.rm = TRUE)
SES <- (obs - mean.null) / sd.null

plot(radii, SES, ylim = range(c(-2, +2, SES)), las = 1)
abline(h = c(-2, +2), lty = 2) # rough guidelines for "strong" effect
```



Going through numerical models does not require calculating areas, which is a time-consuming step. It may be more useful, in general. In terms of controlling for edge effects, the null models have some benefits over expressing things on a per-area basis:

- while the area reflects the average expectation, the null model incorporates variance around that: hypothesis-testing possible
- the null expectation helps also in cases of magnitudes that do not grow linearly with area

The approach taken is very fast but demands a lot of memory, so this can be a practical limitation for large datasets. Working with large data objects is problematic and anything based on distances between pairs tend to be large and time-consuming. The problem is worse when applying null models (1 + 999 times worse). These are some useful things to consider:

- you may not need to store 999 times the large {i, j, distance} null neighborhood object. Instead, it can be temporary within the loop, and you store the relevant data summaries only.
- you may store null expectations into files during the loop, and read and process later.
- if you are going to end summarizing into a SES using mean(null) and sd(null), you can keep updated running values of them using Welford's algorithm. Check it out. It would save the need to store almost anything.
- instances of null models can be run in parallel, if you know how to run code in parallel.

4.2 Example 2: Conspecifics vs. heterospecifics at short distances

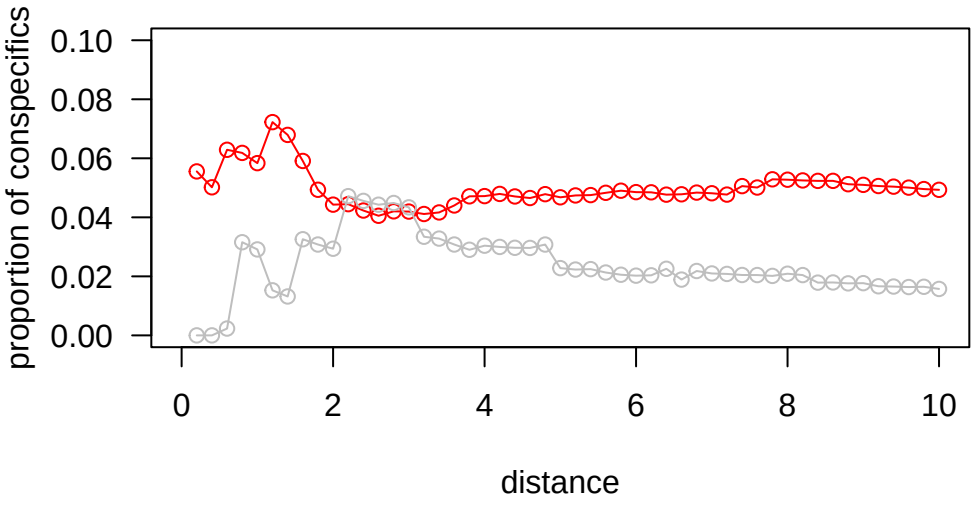
Here, I present a template on how to calculate both conspecifics and heterospecifics at very short distances. This type of calculation may be relevant for those studying negative density dependence, species coexistence, etc. I look for small trees around large trees, as a rough proxy

for “mother trees” and “recent recruits”. It is an example of the type of flexibility you may want to adapt to your case.

```
# We define what we will consider "large" and "small" trees,
# as this will determine both query and reference datasets.
large = 250 # mm
small = 50  # mm

# In this case, we will iterate over smaller query datasets, one per species.
# (It is probably not necessary for the size of our data, but if we can
# use the time to do the loop, it's probably easier to organize and
# safer, from the memory overflow point of view).

# The reference dataset, however, will contain all small trees of all
# the species, because we will be looking at conspecifics and
# heterospecifics. Thus, it can be defined out of the loop
# and used again and again.

keep1 <- data$status == "A"
keep2 <- !is.na(data$gx)
keep3 <- !is.na(data$gy)
keep4 <- data$dbh <= small
keep  <- keep1 & keep2 & keep3 & keep4
reference_data <- data[keep,, drop = FALSE]

# Unexpected problems? Solve them.
#table(is.na(reference_data$gx))
#table(is.na(reference_data$gy))
keep2 <- !is.na(reference_data$gx)
keep3 <- !is.na(reference_data$gy)
keep <- keep2 & keep3
reference_data <- reference_data[keep,, drop = FALSE]

# Now, we define the scales of interest and the target spp:
spp <- unique(data$sp)
radii <- get_radii(rmax = 10, m = 50, case = "circles") # very short distances

# Then, we loop across species:

con <- matrix(0, nrow = length(spp), ncol = length(radii),
              dimnames = list(spp, paste0("radius=", radii))) # empty storage
het <- matrix(0, nrow = length(spp), ncol = length(radii),
              dimnames = list(spp, paste0("radius=", radii))) # empty storage
```

```r
for(sp in spp)
{
  # Define the query data, based on the target species,
  # the large size, living status, valid coordinates, ...
  keep1 <- data$status == "A"
  keep2 <- !is.na(data$gx)
  keep3 <- !is.na(data$gy)
  keep4 <- data$dbh >= large
  keep5 <- data$sp == sp
  keep  <- keep1 & keep2 & keep3 & keep4 & keep5
  query_data <- data[keep,, drop = FALSE]

  # Unexpected problems? Solve them.
  #table(is.na(query_data$gx))
  #table(is.na(query_data$gy))
  keep2 <- !is.na(query_data$gx)
  keep3 <- !is.na(query_data$gy)
  keep <- keep2 & keep3
  query_data <- query_data[keep,, drop = FALSE]

  # Skip if we lack data:
  if(nrow(query_data) == 0) next

  # Obtain neighborhoods:
  neighbors <- get_distances_up_to_r(query = query_data[,c("gx", "gy")],
                                     reference = reference_data[,c("gx", "gy")],
                                     radius = max(radii))

  # There won't be neighbors of themselves if the size thresholds
  # split the data cleanly. But, just in case, the way to remove
  # neighbors of themselves when query =! reference has to get
  # the unique identifyiers first:
  id.in.query <- query_data$treeID[neighbors$query_i]
  id.in.reference <- reference_data$treeID[neighbors$reference_j]
  not.themselves <- id.in.query != id.in.reference
  neighbors <- neighbors[not.themselves,]

  # Get the mean number of neighbors of same or
  # different species, scale by scale:
  n.con <- sapply(radii, function(r) {
    at.distance <- neighbors$dist <= r
    same.sp <- reference_data$sp[neighbors$reference_j] == sp
```

```
    sub <- neighbors[at.distance & same.sp, ]
    mean(tabulate(sub$query_i, nbins = nrow(query_data)))
  })

  n.het <- sapply(radii, function(r) {
    at.distance <- neighbors$dist <= r
    different.sp <- reference_data$sp[neighbors$reference_j] != sp
    sub <- neighbors[at.distance & different.sp, ]
    mean(tabulate(sub$query_i, nbins = nrow(query_data)))
  })

  # Store results and track progress
  con[sp,] <- n.con
  het[sp,] <- n.het
  #print(paste0(which(spp == sp), "/", length(spp)))
}


# Inspect and process results any way you need:
proportion_conspecifics <- con / (con + het)

common <- names(sort(table(data$sp[data$status == "A"]), decreasing = TRUE))[1:20]
rare <- spp[!spp %in% common]

plot(radii, colMeans(proportion_conspecifics[common,], na.rm = TRUE),
     col = "red", type = "o",
     las = 1,
     ylim = c(0, 0.1),
     xlim = range(c(0, radii)),
     ylab = "proportion of conspecifics",
     xlab = "distance")

points(radii, colMeans(proportion_conspecifics[rare,], na.rm = TRUE),
     col = "gray", type = "o")
```




```
# Wow! It seems that the scale at which the proportion of
# conspecifics is LOWEST for common species is the same
# scale at which the proportion of conspecificis is
# the HIGHEST for rare species! Maybe you can write a paper!

# Jokes aside: the % of conspecifics peaks at shorter
# distances for common species than for rare species.
# This seems congruent with general theory.
```

4.3 Example 3: Diversity in neighborhoods

We can think of some species "promoting" diversity more than others, or preferring locations with higher or lower diversity (without assuming direct causal relationship), etc. In this example I calculate diversity indexes in the neighborhood of different species, at different scales.

The example also highlights the use of doing data summaries and aggregation directly from within data.table objects, using `data.table` syntax.

```
# Define query dataset:
keep1 <- data$status == "A"
keep2 <- !is.na(data$gx)
keep3 <- !is.na(data$gy)
keep <- keep1 & keep2 & keep3
query_data <- data[keep,, drop = FALSE]

# Define the reference dataset:
```

```r
# (We are interested in neighbors of any species around any species, so reference = query)
reference_data <- query_data

# Now the scales of interest:
radii <- get_radii(m = 25, rmax = 10) # more informative at closer distances?

# We would be looking at the entire rectangular plot:
window <- c(xmin = 0, ymin = 0, xmax = round(max(data$gx, na.rm = TRUE)), ymax = round(max(da

# Are these datasets too large to cross at once?
n = nrow(query_data)
m = nrow(reference_data)
A = sf::st_area(get_boundary(window = window)) # boundary as sf object, use get_boundary(...)
a = pi*max(radii)^2 # assumes same units as boundary
Q = n * a  * m / A
Q # larger than 10^8?
```

```
[1] 3068975
```

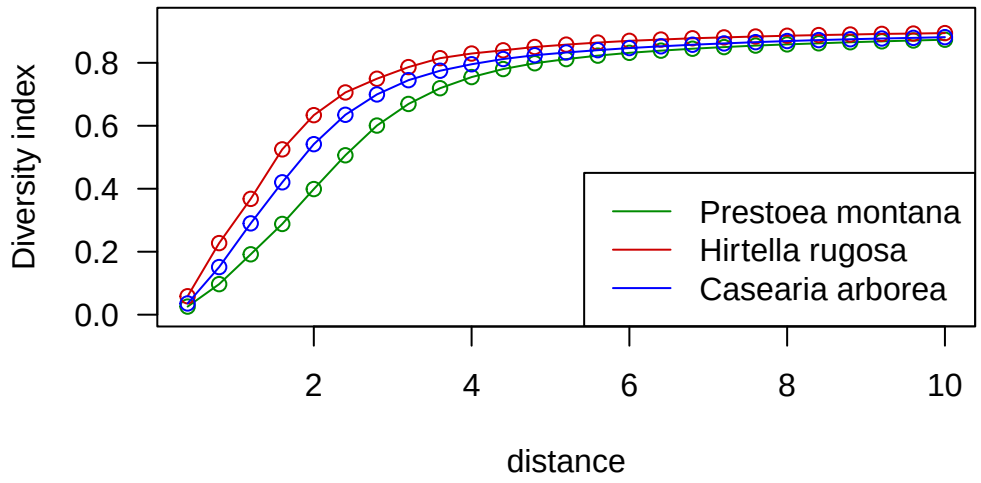
```r
# It seems we will get less than 100 million pairs of neighbors.
# That's not too much. Let's calculate neighbors just once:
neighbors <- get_distances_up_to_r(query = query_data[,c("gx", "gy")],
                                   reference = reference_data[c("gx", "gy")],
                                   radius = max(radii))

# General way of removing neighbors of themselves:
id.in.query <- query_data$treeID[neighbors$query_i]
id.in.reference <- reference_data$treeID[neighbors$reference_j]
not.themselves <- id.in.query != id.in.reference
neighbors <- neighbors[not.themselves,]

nrow(neighbors) / Q # Q was a good guess
```

```
[1] 1.078327
```

```r
# Empty object to store results:
spp <- unique(query_data$sp)
diversity <- matrix(NA, nrow = length(spp), ncol = length(radii), dimnames = list(spp, paste(
# (etc)
```

```r
# Fill by subsetting at different scales:
# Buf, before, we add the species identity for i and j:
neighbors$sp_i <- query_data$sp[neighbors$query_i] # species identity of focal trees: will su
neighbors$sp_j <- reference_data$sp[neighbors$reference_j] # species identity of neighbors j

for(k in 1:length(radii))
{
  # Subset
  radius = radii[k]
  sub <- neighbors[neighbors$dist <= radius, c("query_i", "sp_i", "sp_j")]

  # Use data.table syntax to obtain different diversity indexes:

  # Extract query-level sp_i information, that will be used later to summarize
  query_sp_i <- unique(sub[, .(query_i, sp_i)])

  # Count abundance of each sp_j around each query_i
  comp <- sub[, .(
    abund = .N
  ), by = .(query_i, sp_j)]

  # Total abundance around each query point or focal tree:
  comp[, N := sum(abund), by = query_i]

  # Relative abundance of each species around the query point or focal trees
  comp[, p := abund / N]

  # Diversity around each query point or focal tree
  alpha <- comp[, .(
    N = data.table::first(N),
    richness = .N,
    shannon = -sum(p * log(p)),
    simpson_lambda = sum(p^2)
  ), by = query_i]

  # Derived indices
  alpha[, hill_q0 := richness]
  alpha[, hill_q1 := exp(shannon)]
  alpha[, hill_q2 := 1 / simpson_lambda]
  alpha[, simpson := 1 - simpson_lambda]
  alpha[, invsimpson := 1 / simpson_lambda]
  alpha[, pielou := data.table::fifelse(richness > 1, shannon / log(richness), NA_real_)]
```

```r
  # Add sp_i and then summarize around it:
  alpha <- merge(
    alpha,
    query_sp_i,
    by = "query_i",
    all.x = TRUE
  )

  alpha_by_sp_i <- alpha[, .(
    n_query_i = .N,
    mean_N = mean(N, na.rm = TRUE),
    mean_richness = mean(richness, na.rm = TRUE),
    mean_shannon = mean(shannon, na.rm = TRUE),
    mean_hill_q1 = mean(hill_q1, na.rm = TRUE),
    mean_hill_q2 = mean(hill_q2, na.rm = TRUE),
    mean_simpson = mean(simpson, na.rm = TRUE),
    mean_invsimpson = mean(invsimpson, na.rm = TRUE),
    mean_pielou = mean(pielou, na.rm = TRUE)
  ), by = sp_i]

  # Put in the right place
  # (One index, as an example)
  diversity[alpha_by_sp_i$sp_i, k] <- alpha_by_sp_i$mean_simpson
  #print(paste0(k, "/", length(radii)))
}


# Then you visualize and process as you want
YLIM = range(diversity, na.rm = TRUE)
plot(radii, diversity["PREMON", ], type = "o", col = "green4", ylim = YLIM, ylab = "Diversity
points(radii, diversity["HIRRUG", ], type = "o", col = "red3")
points(radii, diversity["CASARB", ], type = "o", col = "blue")
legend("bottomright", legend = c("Prestoea montana", "Hirtella rugosa", "Casearia arborea"),
```



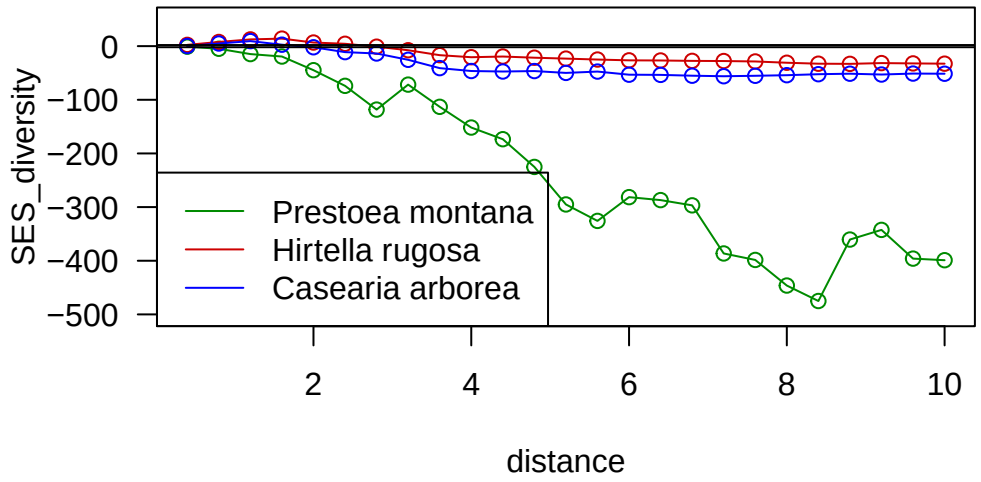
Diversity is something that does not grow linearly with area, so there is no need to calculate the area of the neighborhoods. In fact, that would be misleading.

We can use null models to express diversity as "deviations from the expected", which would directly correct for edge effects. For simplicity, I use the simple null model of random distribution – this, by no means, is a general recommendation for studies of local diversity. The model is equivalent to permuting species identities.

```
nsim = 9 # 999 for real applications
null <- array(NA, dim = c(length(spp), length(radii), nsim),
              dimnames = list(spp, NULL, NULL))

for(sim in 1:nsim)
{
  # Null reference xy, assuming uniform distribution
  m = nrow(reference_data)
  xmin = floor(min(reference_data$gx, na.rm = TRUE))
  xmax = ceiling(max(reference_data$gx, na.rm = TRUE))
  ymin = floor(min(reference_data$gy, na.rm = TRUE))
  ymax = ceiling(max(reference_data$gy, na.rm = TRUE))
  null.x <- runif(m, min = xmin, max = xmax)
  null.y <- runif(m, min = ymin, max = ymax)
  null.reference.xy <- cbind(null.x, null.y)

  # Null neihgbors:
  null.nnn <- get_distances_up_to_r(query = query_data[,c("gx", "gy")],
                                    reference = null.reference.xy,
                                    radius = max(radii))
```

```
  # General way of removing neighbors of themselves:
  id.in.query <- query_data$treeID[null.nnn$query_i]
  id.in.reference <- reference_data$treeID[null.nnn$reference_j]
  not.themselves <- id.in.query != id.in.reference
  null.nnn <- null.nnn[not.themselves,]

  # Add species identities
  null.nnn$sp_i <- query_data$sp[null.nnn$query_i]
  null.nnn$sp_j <- reference_data$sp[null.nnn$reference_j]

  # Loop through scales for the null neighborhoods:
  for(k in 1:length(radii))
  {
    # Subset
    radius = radii[k]
    sub.null <- null.nnn[null.nnn$dist <= radius, c("query_i", "sp_i", "sp_j")]

    # Repeat the same things as above, but for the null neighborhoods
    null.query_sp_i <- unique(null.nnn[, .(query_i, sp_i)])
    null.comp <- sub.null[, .(abund = .N), by = .(query_i, sp_j)]
    null.comp[, N := sum(abund), by = query_i]
    null.comp[, p := abund / N]
    null.alpha <- null.comp[, .(simpson_lambda = sum(p^2)), by = query_i]
    null.alpha[, simpson := 1 - simpson_lambda]

    null.alpha <- merge(
      null.alpha,
      null.query_sp_i,
      by = "query_i",
      all.x = TRUE
    )

    null.alpha_by_sp_i <- null.alpha[, .(mean_simpson = mean(simpson, na.rm = TRUE)),
                                     by = sp_i]

    # Store in the right place:
    null[null.alpha_by_sp_i$sp_i,k,sim] <- null.alpha_by_sp_i$mean_simpson
  }

  # track
  #print(paste0(sim, "/", nsim))
}
```

```r
# Then, compare observations with expectations:
mean.null <- apply(null, c(1, 2), FUN = mean, na.rm = TRUE)
sd.null   <- apply(null, c(1, 2), FUN = sd, na.rm = TRUE)
sd.null[which(sd.null == 0)] <- min(sd.null[which(sd.null > 0)], na.rm = TRUE)
SES_diversity <- (diversity - mean.null) / sd.null

# Etc.

YLIM = c(-500, +50)
plot(radii, SES_diversity["PREMON", ], type = "o", col = "green4", ylim = YLIM, ylab = "SES_d
points(radii, SES_diversity["HIRRUG", ], type = "o", col = "red3")
points(radii, SES_diversity["CASARB", ], type = "o", col = "blue")

legend("bottomleft", legend = c("Prestoea montana", "Hirtella rugosa", "Casearia arborea"), l

abline(h = c(-2, +2)) # guidelines for strong effect
```




```r
# Note: the null model is a complete mix, so it creates highly diverse
# communities at almost every scale. Observed neighborhoods tend to be
# way less diverse than expected, under this extreme expectation.
# If you really want to learn about local diversity in neighborhoods,
# then you really need to think about better null models than this!
```

5 Legacy notes

5.1 Old CTFS functions

The code substitutes these original CTFS functions:

- `CalcRingArea` in spatial > RipUvK.r
- `partialcirclearea` in spatial > RipUvK.r
- `circlearea` in spatial > RipUvK.r
- `Annuli` in spatial > RipUvK.r
- `NDcount` in spatial > NeighborDensityFun.r
- `NeighborDensities` in spatial > NeighborDensityFun.r
- `RipUvK` in spatial > RipUvK.r

The old functions were closer to "plug data in and forget" than the current functions. The cost was very limited flexibility. Now, filters, abundance metrics, null expectations, and classes to compare, are all outside the functions. This puts more power = responsibility on the hands of the users.

All area-related tasks are handled by the `get_area_of_neighborhoods()` function now, covering circles and rings, and arbitrary boundary shapes.

The neighbor-counting CTFS functions implemented common ways of aggregating data. The functions made many assumptions about classes to include, units, etc. and also use obsolete spatial calculations for boundaries, etc. Neighbor-counting is just one instance of the general task of calculating *any* metric of interest over a set of neighbors. The current code does not attempt to solve that type of task in general. I have provided guidelines and templates instead. You will have to adapt the examples to your case, and find efficient ways of aggregating or summarizing data for your case.

Finally, the Ripley's K is not implemented at all. It is already implemented in many R package, probably dozens of them. Because this is a standard tool, here is a worked example below on how to do that type of analysis:

5.2 Example: Ripley's K and Besag's L

Ripley's K function measures how many neighboring points occur within distance r, relative to what is expected under complete spatial randomness. Values above expectation indicate clustering; values below expectation indicate regularity or inhibition. Besag's L is a variance-stabilized transformation of K, often plotted as L(r) − r, making deviations from randomness easier to interpret.

```r
# We use one species as a example
sp = "PREMON" # Prestoea, the most common palm

# The gold standard is in spatstat packages:
library(spatstat.geom)
```

```
Warning: package 'spatstat.geom' was built under R version 4.4.3
```

```
Loading required package: spatstat.data
```

```
Loading required package: spatstat.univar
```

```
Warning: package 'spatstat.univar' was built under R version 4.4.3
```

```
spatstat.univar 3.2-0
```

```
spatstat.geom 3.8-1
```

```r
library(spatstat.explore)
```

```
Warning: package 'spatstat.explore' was built under R version 4.4.3
```

```
Loading required package: spatstat.random
```

```
Warning: package 'spatstat.random' was built under R version 4.4.3
```

```
spatstat.random 3.5-0
```

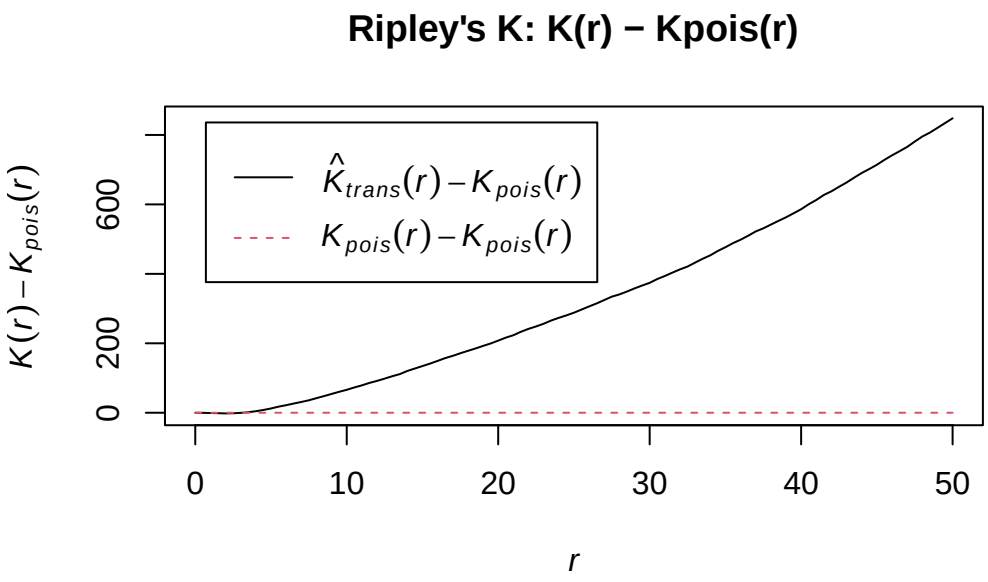
```
Loading required package: nlme
```

```
spatstat.explore 3.8-1
```

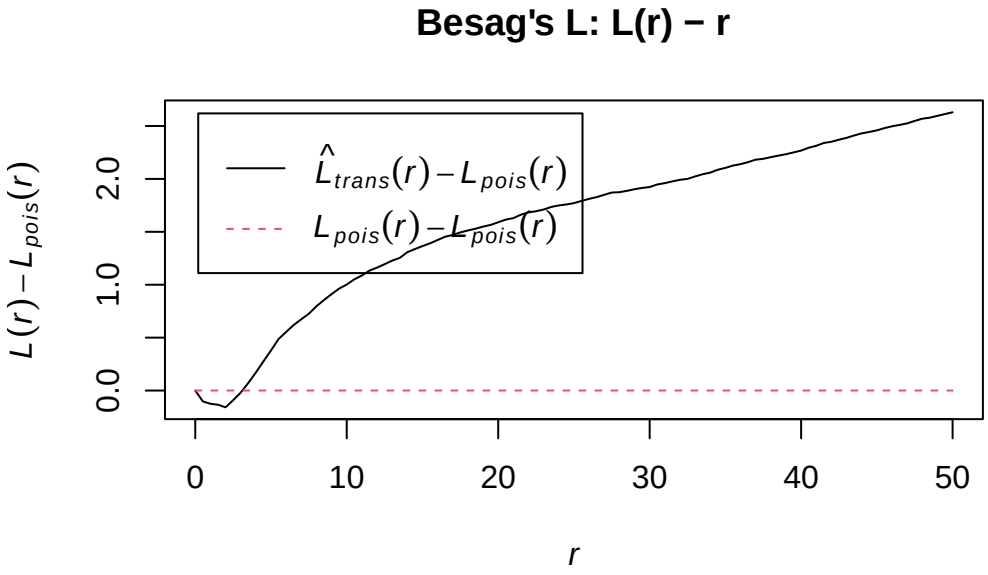
```r
# This is the rectangular observation window:
W <- owin(xrange = round(range(data$gx, na.rm = TRUE)),
          yrange = round(range(data$gy, na.rm = TRUE)))

# Point pattern spatial object for the species;
# we focus only on the living trees.
x <- data$gx[data$sp == sp & data$status == "A"]
y <- data$gy[data$sp == sp & data$status == "A"]
PPP <- ppp(x = x, y = y, window = W)
```

```
Warning in ppp(x = x, y = y, window = W): 56 out of 6014 points had NA or NaN
coordinate values, and were discarded
```

```r
# Aggregation at different scales:
radii <- seq(from = 0, to = 50, by = 0.5)
K <- Kest(PPP, r = radii, correction = "translate")
L <- Lest(PPP, r = radii, correction = "translate")

# Results:
plot(K, . - theo ~ r, main = "Ripley's K: K(r) - Kpois(r)")
```




```r
plot(L, . - theo ~ r, main = "Besag's L: L(r) - r")
```




```
# With simulation envelopes, only for Besag's L,
# which is easier to interpret visually:
L.env <- envelope(PPP, fun = Lest, r = radii, nsim = 199, verbose = FALSE)
plot(L.env, . - theo ~ r, main = "Global CSR envelope: L(r) - r")
```